

ShiftOps: Enterprise Hospital Workforce Shift Scheduling & Compliance SaaS Platform

Course: Software Architecture
Course Code : CSE 4435
Section: A

Submitted BY
Md. Mahin Khan - 0112230105
Md. Asaduzzaman - 0112230091
Md. Noman Hossain - 011222159

Submitted To
Tanmoy Bipro Das
Lecturer, Dept. of CSE

Date of submission
10th June, 2025

Table of Content

1. Introduction....2
2. Analysis....3
2.1. Tactics-Based Questionnaire for Availability Analysis.... 3
2.2. Tactics-Based Questionnaire for Deployability Analysis.... 7
2.3. Tactics-Based Questionnaire for Energy Efficiency Analysis....10
3. Utility Tree.... 13
4. Chosen Architectural Pattern: Microservices....16
4.1. Why are Microservices the Best Fit for Our Project?....16
4.2. Additional Justification.... 17
4.3. Why Did We Not Choose Other Patterns?.... 18
4.4. Summary.... 19
5. Application of the Chosen Architecture Pattern....19
6. Architecture Diagram: Applied Microservices for ShiftOps....20
Diagram.... 20
6.1. System Architecture Diagram Description....22
6.2. Component Breakdown (Applied to ShiftOps)....22
7. How the Pattern Works in Our Case.... 24
7.1. Pattern Analysis....26
Overall Agility....26
Ease of Deployment....26
Scalability....26
Performance (Latency & Throughput)....27
Maintainability.... 27
Energy Efficiency.... 27
Security & Confidentiality....28
Availability....28
Testability.... 28
Conclusion.... 29

1. Introduction

ShiftOps is a multi-tenant Software-as-a-Service (SaaS) platform designed to address one of the most operationally critical challenges in hospital management: ensuring that the right staff are scheduled for the right shifts, at all times, while remaining fully compliant with labor regulations — and allowing controlled emergency overrides when patient safety demands it.

In hospital environments, shift scheduling failures are not mere inconveniences. An unstaffed ward, a nurse scheduled beyond legal working hour limits, or a scheduling system that goes offline at 3 AM are scenarios with direct consequences for patient safety, staff wellbeing, and regulatory standing. ShiftOps is built with these stakes in mind.

The platform serves hospitals as tenants. Each hospital subscribes to ShiftOps and receives a fully isolated environment containing its own staff roster, department structure, shift rules, and compliance configurations. Within each tenant, four primary user roles interact with the system:

- **HR Administrator** — manages staff records, approves leave, authorizes emergency overrides, configures compliance rules
- **Department Manager** — creates and publishes shift schedules, requests emergency overrides, monitors department coverage
- **Staff Member (Nurse/Doctor)** — views personal schedule, submits availability and leave requests, acknowledges shift assignments
- **Compliance Auditor** — reviews compliance reports, queries audit trails, exports regulatory documentation

ShiftOps resolves the central tension of hospital scheduling: labor laws must be enforced (staff safety, legal compliance) but emergencies override everything (patient safety, operational continuity). The system implements a **hard/soft constraint model**. Hard constraints are never violable. Soft constraints block by default but can be overridden by an authorized role with mandatory justification, producing an immutable audit record — the **override-with-audit pattern**.

Core capabilities include automated shift generation via Google OR-Tools constraint optimization, real-time compliance validation against Bangladesh Labour Act 2006, emergency override management with full audit logging, payroll summary export for integration with existing hospital payroll systems, and instant staff notifications on all schedule events.

---

2. Analysis

2.1. Tactics-Based Questionnaire for Availability Analysis

Availability is the most critical quality attribute for ShiftOps. A hospital scheduling system must remain operational 24/7. Downtime during a shift transition leaves wards unable to confirm staffing, directly threatening patient safety.

Tactics Group	Support ? (Y/N)	Risk	Design Decision and Location	Rationale and Assumptions
Fault Detection — Heartbeat / Health Monitoring	Y	Low	Each microservice exposes a /health endpoint. The API Gateway polls all services every 30 seconds and removes unresponsive instances from the routing pool automatically.	Assumes container orchestration (e.g., Docker Compose or Kubernetes) restarts failed containers. Without active health checks, a silently failed service continues receiving routed traffic and dropping requests.
Fault Detection — Exception Monitoring	Y	Low	All services emit structured error logs to a centralized logging sink. Critical errors (Scheduling Service crash, DB connection failure) trigger an alert to the on-call system administrator.	Assumes a log aggregator is configured. Unmonitored exceptions accumulate silently and cause cascading failures. Structured logging (JSON format) is required for reliable parsing.

Fault Recovery — Automatic Restart	Y	Low	Failed service containers are configured with restart policies (restart: always). The orchestrator restarts a failed instance within 10 seconds with no manual intervention.	Assumes the failure is transient (crash-loop scenarios require human intervention). Restart policies alone do not cover data corruption failures — those require rollback.
Fault Recovery — Redundant Instances	Y	Medium	The Scheduling Service and API Gateway are deployed with a minimum of two active instances behind a load balancer. Requests are rerouted to healthy instances if one fails.	Assumes a load balancer is present. Single-instance critical services are a single point of failure. The medium risk reflects the cost and complexity of maintaining multiple instances in a resource-constrained environment.
Fault Isolation — Bulkhead Pattern	Y	Medium	Each microservice runs in its own isolated container with defined CPU and memory limits. A memory leak or crash in the Payroll Summary Service cannot consume resources from the Scheduling Service.	Assumes resource limits are configured correctly at the container level. Without limits, a runaway service can starve other services of resources even in separate containers.

Graceful Degradation	Y	Medium	If the Scheduling Engine (OR-Tools optimization) is offline, the system falls back to read-only mode: staff can view published schedules but new generation is queued. A degradation banner is shown to managers.	Assumes the read path and write path are architecturally separated. If they share the same service instance, a crash in the optimization thread can take down schedule viewing as well.
Saga Pattern — Compensating Transactions	Y	High	Schedule publication is a multi-step operation across Scheduling Service, Compliance Service, and Audit Service. If publication fails mid-way, compensating transactions revert all partial changes within 5 seconds.	High risk because distributed transaction management is complex. Assumes each service exposes a compensation endpoint. Failure to implement compensating transactions results in orphaned partial schedule states that corrupt the system view.

State Resynchronization	Y	Low	On service restart after failure, the service re-reads its last confirmed state from the database before resuming operations. In-flight requests at time of crash are retried by the client after a timeout.	Assumes idempotent API design — retried requests must not create duplicate records. All write operations include idempotency keys to prevent duplicate schedule entries.

2.2. Tactics-Based Questionnaire for Deployability Analysis

Deployability is essential for ShiftOps because labor compliance rules change when legislation is amended, scheduling logic evolves with product updates, and tenant onboarding must happen at runtime — all without system downtime.

Tactics Group	Support ? (Y/N)	Risk	Design Decision and Location	Rationale and Assumptions
Independent Deployability — Per-Service CI/CD	Y	Low	Each microservice has its own CI/CD pipeline. The Compliance Service can be built, tested, and deployed to production independently of all other services. A compliance rule update does not trigger redeployment of the Scheduling Service.	Assumes services are genuinely decoupled — no shared runtime libraries that force synchronized version bumps. A shared library version change that affects multiple services defeats independent deployability.

API Versioning	Y	Medium	All inter-service REST contracts are versioned with URL prefixes (/v1/, /v2/). Old and new versions run simultaneously during transition periods. Services continue to call the previous version until explicitly updated.	Medium risk because maintaining multiple live API versions increases operational overhead. Assumes a deprecation timeline is enforced — indefinite version accumulation creates a maintenance burden.
Blue-Green Deployment	Y	Medium	New service versions are deployed to an idle environment (green). Traffic switches from the active environment (blue) to green only after health checks pass. Rollback is instant — switch traffic back to blue if the green deployment fails.	Requires double the infrastructure resources during deployment windows. Assumes the deployment pipeline automates the traffic switch. Manual switches introduce human error risk.
Configuration-Driven Compliance Rules	Y	Low	Bangladesh Labour Act 2006 rules are stored as tenant configuration records in the database, not hardcoded in the Compliance Service. Rule updates (e.g., a change to the maximum weekly hours threshold) are applied via admin panel with no code redeployment.	Assumes the rule engine in the Compliance Service is general enough to interpret any configured rule. If new rule types require new code logic (not just new thresholds), a redeployment is still needed.

Expand-Contract Schema Migrations	Y	Medium	Database schema changes follow the expand-contract pattern: new columns are added as nullable first (expand), both old and new service versions run simultaneously, then old columns are deprecated and removed in a later migration (contract).	Medium risk because the expand phase temporarily allows inconsistent data states. Assumes the data access layer handles nullable new columns gracefully without crashing on null reads.
Rollback Capability	Y	Low	Each service deployment retains the previous container image. A one-command rollback redeploys the previous image and points the load balancer back to it within 60 seconds.	Assumes the previous image is retained in the container registry and database migrations are backward-compatible. A migration that drops a column cannot be rolled back by reverting the service image alone.
Runtime Tenant Provisioning	Y	Low	New hospital onboarding is a fully automated runtime operation. The Tenant Management Service creates a new PostgreSQL schema, runs the full migration set, and seeds default compliance rules without any manual deployment step.	Assumes the provisioning service has sufficient PostgreSQL permissions to create schemas dynamically. Security review required to ensure tenant schemas cannot reference or access each other's data.

Feature Flags	Y	Low	New features (e.g., a new shift type, a new notification channel) are deployed behind feature flags that are off by default and enabled per tenant via admin configuration.	Allows gradual rollout and instant disabling of a broken feature without redeployment. Assumes a feature flag store (Redis or DB table) is consulted at service startup or request time.

2.3. Tactics-Based Questionnaire for Energy Efficiency Analysis

Energy efficiency is relevant to ShiftOps because constraint-solving optimization is computationally expensive, the system runs on cloud infrastructure with real cost implications, and 24/7 operation creates opportunities for intelligent resource management during low-activity periods.

Tactics Group	Support ? (Y/N)	Risk	Design Decision and Location	Rationale and Assumptions
Elastic Scaling — Scale to Zero	Y	Medium	The Scheduling Engine (OR-Tools service) scales to zero replicas when no optimization jobs are in the queue. It spins up on-demand when a generation request arrives. This eliminates idle compute cost during off-peak hours.	Medium risk because scale-to-zero introduces cold start latency (5–15 seconds to spin up). Assumes hospital managers do not require instant schedule generation — a queued job model with async result notification is acceptable.

Scheduled Batch Processing	Y	Low	Monthly and weekly schedule generation for full departments is triggered as a batch job at off-peak hours (default: 2:00 AM local time). This concentrates expensive OR-Tools computation during minimum-traffic periods.	Assumes the hospital's scheduling cycle accommodates advance planning. Emergency ad-hoc generation remains available on-demand at higher resource cost. Batch scheduling is the primary energy efficiency lever.
Result Caching	Y	Low	Compliance Service validation results are cached in Redis with the schedule version as the cache key. If a schedule has not changed since the last validation, the cached result is returned with no recomputation.	Assumes schedule versions are immutable once validated. Cache invalidation is triggered only when the schedule content changes. TTL set to 24 hours to prevent stale compliance results on rule updates.
Asynchronous Audit Logging	Y	Low	Audit events are published to the Redis message queue and consumed asynchronously by the Audit Service. The main request thread is not blocked waiting for audit writes, reducing CPU time per request.	Assumes the message queue provides at-least-once delivery guarantees. Audit records must include idempotency keys to prevent duplicate log entries from redelivered messages.

Read Replica for Schedule Queries	Y	Medium	Schedule view queries (the highest frequency read operation) are routed to a PostgreSQL read replica. Only write operations (schedule creation, publication, overrides) target the primary database.	Medium risk because read replicas have replication lag (typically <1 second). A manager who publishes a schedule and immediately views it might see the pre-publication version for a brief window. Assumes this is acceptable for the use case.
Batched Notification Dispatch	Y	Low	When a schedule is published, affected staff notifications are grouped by department and dispatched as a batch rather than N individual API calls. A 50-person department generates one batch notification dispatch, not 50 sequential calls.	Requires the Notification Service to support batch recipient lists. Reduces outbound API calls to email/push notification providers, lowering both latency and external API cost.
Shared Infrastructure Across Tenants	Y	Low	All tenant workloads share the same service instances. Only data is isolated via PostgreSQL schemas. A single Scheduling Service instance serves all hospitals, eliminating idle per-tenant resource waste.	Assumes tenant workloads are not simultaneously intensive. A pathological case where all hospitals run batch generation simultaneously would saturate shared resources. Mitigated by staggering batch job start times per tenant.

Idle Connection Pooling	Y	Low	Database connections are managed via a connection pool (PgBouncer). Idle connections are returned to the pool rather than held open. Pool size is tuned per service based on expected concurrency.	Without connection pooling, each request opens and closes a database connection, creating connection overhead and exhausting PostgreSQL's connection limit under load.

---

3. Utility Tree

Quality Attribute	Attribute Refinement	ASR Scenario	Priority (H/M/L)
Availability	Fault Recovery	The Scheduling Service crashes at 3 AM during an active shift transition. The system detects the failure via health check within 30 seconds, restarts the service container automatically, and resumes pending requests without data loss. The on-call administrator receives an alert.	H
	Graceful Degradation	The OR-Tools optimization engine is offline due to a dependency failure. Staff members and managers can still view all previously published schedules in read-only mode. New schedule generation requests are queued and processed when the engine recovers.	H
	Distributed Transaction Integrity	A schedule publication operation fails after the Scheduling Service writes the record but before the Audit Service logs the event. Compensating transactions automatically revert the partial write within 5 seconds, leaving the system in a consistent state.	H

Deployability	Independent Service Release	A new amendment to Bangladesh Labour Act 2006 changes the maximum weekly hours threshold. Only the Compliance Service is updated and redeployed. All other services continue running uninterrupted. The deployment completes within 10 minutes.	H
	Zero-Downtime Deployment	A new version of the Notification Service is released with a new email template engine. Blue-green deployment switches traffic to the new version with no interruption to active user sessions. Rollback completes in under 60 seconds if the new version fails health checks.	M
	Runtime Tenant Onboarding	A new hospital signs up for ShiftOps. Their isolated PostgreSQL schema is provisioned, default compliance rules are seeded, and the initial HR Administrator account is created within 60 seconds of registration, with no manual intervention and no impact on existing tenants.	M
Energy Efficiency	Elastic Scaling	No shift optimization jobs are queued for 4 consecutive hours during a weekend. The Scheduling Engine automatically scales to zero replicas, eliminating idle compute cost. It spins back up within 15 seconds when the next job arrives.	M
	Off-Peak Batch Processing	A department manager requests a monthly schedule generation for a 200-person department. The job is queued and executed at 2:00 AM, completing within 10 minutes. The manager receives an in-app notification when the draft schedule is ready for review.	M

Security	Tenant Data Isolation	A database query issued in the context of Hospital A cannot retrieve, modify, or detect the existence of data belonging to Hospital B. Isolation is enforced at both the application layer (tenant_id middleware) and the database layer (schema-per-tenant).	H
	Override Accountability	A Department Manager requests an emergency override of the 11-hour rest constraint at 2 AM. The system forces entry of a mandatory reason and incident reference number. The override is logged immutably with the authorizing user, timestamp, violated rule, reason, and reference. The Compliance Auditor is notified.	H
Modifiability	Configurable Compliance Rules	A hospital wants to add a custom internal rule prohibiting ICU nurses from working consecutive night shifts. The rule is configured via the admin panel without any code change or service redeployment. The rule takes effect on the next schedule generation.	M
Performance	Schedule View Response Time	A staff member opens their weekly schedule view. The response is returned within 500ms under normal operating load (up to 500 concurrent users). Read queries are served from the PostgreSQL read replica to reduce primary database load.	M
Testability	Service Isolation for Testing	The Compliance Service can be unit-tested in complete isolation by injecting mock schedule data without requiring a running Scheduling Service, database, or message bus. Each service exposes a well-defined interface that supports test doubles.	L

4. Chosen Architectural Pattern: Microservices

4.1. Why are Microservices the Best Fit for Our Project?

Report Evidence	Quality-Attribute Need	How Microservices Satisfies It
Hospital scheduling operates 24/7. A system outage during a shift transition directly endangers patients.	Availability — fault isolation and independent recovery	Each service runs in its own container. A Payroll Summary Service failure cannot affect the Scheduling Service. Failures are structurally contained, not just handled in code.
Labor law rules change when legislation is amended. Compliance updates must not require full system redeployment.	Deployability — independent release of compliance logic	The Compliance Service has its own CI/CD pipeline and deploys independently. A rule update ships without touching or retesting any other service.
OR-Tools constraint optimization is computationally expensive. Notification dispatch is I/O-bound. Audit logging is always-on but low-volume.	Energy Efficiency — per-service elastic scaling	Each service scales according to its own profile. The Scheduling Engine scales to zero when idle; the Notification Service scales horizontally after a mass publish event; the Audit Service runs on minimal fixed resources.
Emergency overrides require a separate authorization flow distinct from normal scheduling operations.	Security — role-scoped authorization with audit trail	The override-with-audit flow is implemented in a dedicated path in the Compliance Service and Auth Service. Authorization logic is not scattered across a monolithic codebase — it is centralized in services with clear ownership.
New hospitals must be onboarded at runtime with isolated data environments, with no redeployment and no impact on existing tenants.	Deployability + Security — runtime provisioning and tenant isolation	The Tenant Management Service provisions new schemas dynamically. Schema-per-tenant isolation is enforced structurally at the database level, not just by application-layer filtering.

Schedule viewing, compliance validation, and authentication require immediate responses. Notifications and audit logging do not.	Performance — synchronous where needed, async where appropriate	Microservices enables a hybrid communication model: synchronous REST for request-response flows, asynchronous Redis Pub/Sub for notifications and audit logging. A monolith cannot make this distinction cleanly.
The system must produce a complete, queryable audit trail of all scheduling events and overrides for regulatory compliance.	Security + Modifiability — dedicated audit capability	A dedicated Audit Service consumes all events from the message bus independently. Adding a new auditable event type requires only a new event publisher and a new consumer handler — no changes to the core scheduling logic.

4.2. Additional Justification

- **Natural bounded contexts:** ShiftOps decomposes cleanly into seven bounded contexts — Auth, Scheduling, Compliance, Notification, Payroll Summary, Audit, and Tenant Management. These are not artificial divisions; they reflect genuinely independent business capabilities with different domain models, data ownership, and lifecycles. Microservices formalizes these natural boundaries into deployment units.
- **Different technology requirements per service:** The OR-Tools optimization engine runs best as a Python service (OR-Tools has native Python bindings). All other services use Node.js/Express. Microservices allows this polyglot approach — a monolith would force a single technology choice or create an awkward mixed-language codebase.
- **Override-with-audit pattern requires service collaboration:** The emergency override flow requires the Compliance Service to detect the violation, the Auth Service to verify the authorizing role, and the Audit Service to log the event. In microservices, each service has a clear responsibility in this flow. In a monolith, the same logic would be entangled across modules with no clear ownership.
- **Tenant isolation is structural, not just logical:** Schema-per-tenant isolation means each hospital's data lives in a separate PostgreSQL schema. This structural isolation is enforced at the database level, not just by application-layer WHERE tenant_id = ? filters. A bug in the application layer cannot accidentally expose cross-tenant data because the schema boundary exists independently of the application.
- **Compliance reporting and operational scheduling have different SLAs:** The Scheduling Service needs sub-second response times for real-time operations. The Compliance Audit reports are batch-generated, can tolerate minutes of processing time, and are accessed infrequently. Separating them into independent services allows each to be optimized for its own SLA without compromising the other.

4.3. Why Did We Not Choose Other Patterns?

Pattern	Rejection Rationale (Based on Report Requirements)
Monolithic Architecture	A monolith fails all three key quality attributes identified in the analysis. (1) Availability: a crash in the payroll module takes down scheduling — fault isolation is impossible in a single process. (2) Deployability: a compliance rule update requires redeploying and retesting the entire application, creating unnecessary risk and downtime. (3) Energy Efficiency: the entire application must scale as a unit — the expensive OR-Tools solver cannot be scaled independently from the lightweight audit logger.
Layered (N-Tier) Architecture	A layered architecture improves code organization over an unstructured monolith but remains a single deployable unit. It fails the deployability requirement for the same reason as a monolith: updating the compliance layer requires redeploying the entire system. It also provides no structural fault isolation — an unhandled exception in the business logic layer can crash all layers simultaneously. The tactics-based analysis requires independent deployment and independent fault containment, which layered architecture cannot provide.
Pure Event-Driven Architecture	A purely event-driven system routes all operations through asynchronous messaging. ShiftOps has multiple operations requiring synchronous request-response semantics: JWT validation must return immediately before a request is processed; compliance validation must return a pass/fail result before a schedule is published; schedule viewing must return data within 500ms. Forcing these operations through an async event bus introduces unnecessary latency and complexity. ShiftOps uses event-driven communication selectively — for notifications and audit logging — but not as the universal communication model.
Serverless Architecture	Serverless functions have two properties incompatible with ShiftOps requirements. (1) Cold start latency: a function that has not been invoked recently takes 5–15 seconds to start. For a hospital manager viewing a schedule or an emergency override confirmation, this latency is unacceptable. (2) Stateless execution: the OR-Tools constraint solver maintains internal state during a solve session. Serverless functions are stateless by design and cannot host a long-running optimization process reliably.

Pipe-and-Filter Architecture	Pipe-and-filter is well-suited to data transformation pipelines with sequential processing stages. ShiftOps is not a data transformation system — it is an interactive, multi-user operational platform with real-time responses, role-based authorization, and complex business logic. The pattern provides no mechanisms for the fault isolation, independent deployability, or elastic scaling identified as critical in the tactics analysis.

4.4. Summary

The microservices architectural pattern was selected for ShiftOps because it is the only pattern that simultaneously satisfies all three quality attributes identified as critical in the tactics-based analysis.

Availability is satisfied structurally: each service is an independent failure domain. A failure in one service cannot cascade to others because there is no shared process space. Health monitoring, automatic restart, and redundant instances are applied per service, giving the system fine-grained recovery control that no single-process architecture can match.

Deployability is satisfied through independent CI/CD pipelines per service. The compliance rule update scenario — the most business-critical deployability requirement — is directly enabled by the Compliance Service being a separate, independently deployable unit. No other pattern in the comparison provides this capability.

Energy efficiency is satisfied through per-service elastic scaling. The Scheduling Engine, the most computationally expensive component, can scale to zero when idle and scale up only during batch optimization runs. This is only possible because it is a separate service with its own resource allocation — not a module inside a shared application process.

The selection of microservices is not a default choice or an academic preference. It is a direct response to the specific operational reality of a 24/7 hospital platform where independent deployment of compliance logic, structural fault isolation, and elastic scaling of expensive computation are genuine requirements, not optional optimizations.

---

5. Application of the Chosen Architecture Pattern

The Microservices pattern is applied to ShiftOps by decomposing the system into seven independent services. Each service owns a single bounded context, manages its own data via a dedicated PostgreSQL schema (per tenant), and operates on an independent deployment lifecycle. Services communicate synchronously via REST for operations requiring immediate responses, and asynchronously via Redis Pub/Sub for event-driven operations where the caller does not need to wait for the result.

The multi-tenancy model is **schema-per-tenant in PostgreSQL**. When a new hospital registers, the Tenant Management Service automatically provisions a new schema and runs the full migration set against it. All subsequent service requests include a tenant_id claim extracted from the JWT at the API Gateway. This claim is injected as a request header into every downstream service call. Each service prefixes its database connection with the tenant's schema name before executing any query, ensuring complete data isolation without requiring separate database servers per tenant.

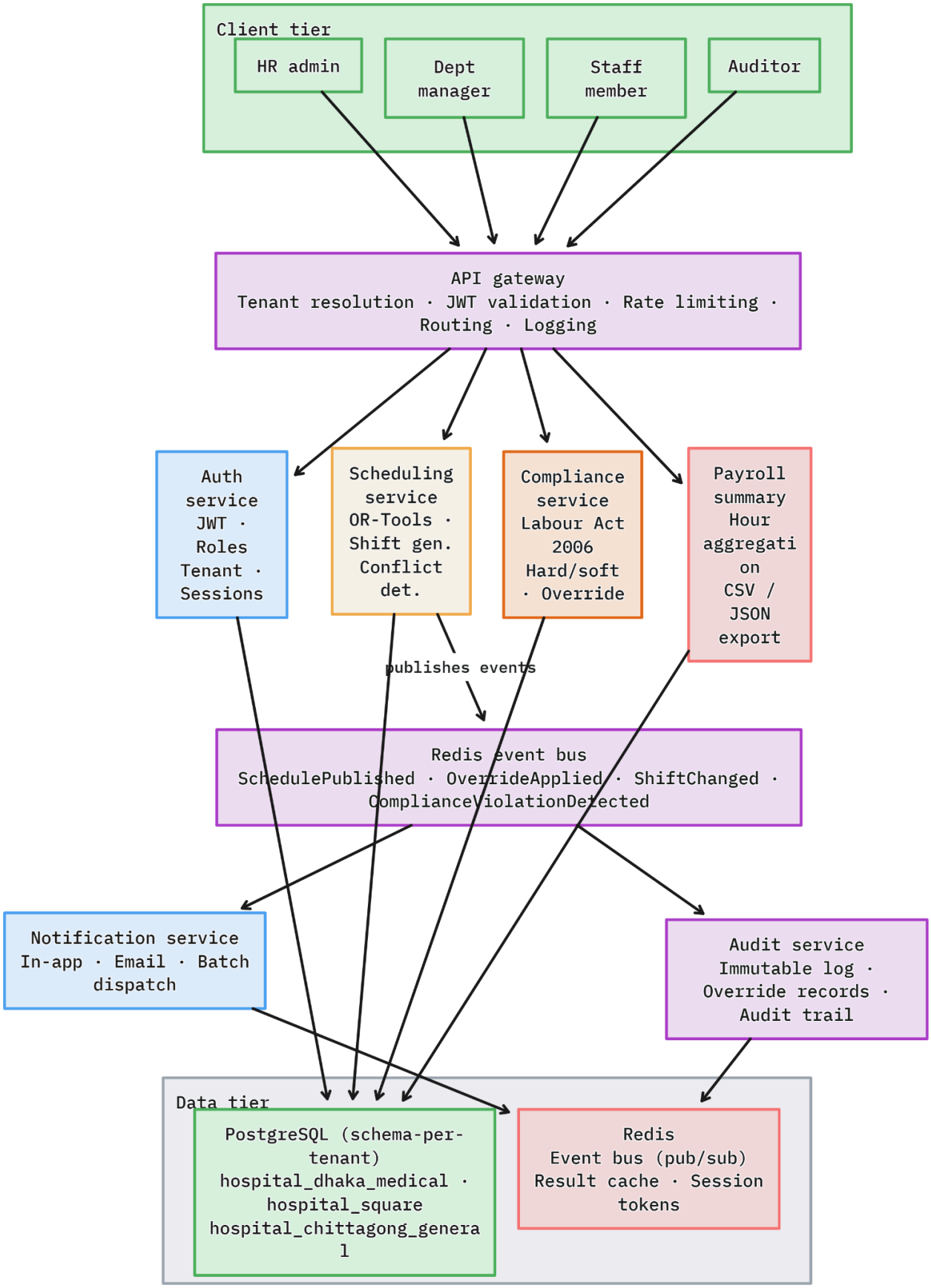
Key application decisions:

- **API Gateway as the single entry point:** All external traffic enters through the API Gateway. No service is directly accessible from outside the platform. This centralizes tenant resolution, authentication validation, and rate limiting in one place, preventing duplication across services.
- **Compliance Service as a pre-publication gate:** The Scheduling Service never publishes a schedule without first receiving a PASS response from the Compliance Service. This enforces the compliance validation requirement as an architectural constraint, not just a code convention.
- **Hard/Soft constraint enforcement in Compliance Service:** The Compliance Service returns three-tier responses — PASS, SOFT_VIOLATION (overridable by authorized role), HARD_VIOLATION (blocked unconditionally). This distinction is encoded in the service's response contract, making it impossible for a calling service to accidentally skip the authorization check for an override.
- **Event bus for fire-and-forget operations:** After a schedule is published, the Scheduling Service emits a SchedulePublished event to Redis. The Notification Service and Audit Service consume this event independently. The Scheduling Service does not wait for notifications to be sent or audit records to be written before returning success to the manager. This keeps the synchronous request path fast while ensuring all downstream effects eventually occur.
- **Override-with-audit as a first-class flow:** Emergency overrides are not handled as a special case in the Scheduling Service. They follow a dedicated API path that requires an explicit override payload (reason, incident reference, authorizing user). The Compliance Service validates the override authorization, and the Audit Service receives a high-priority OverrideApplied event with the full override context.

---

6. Architecture Diagram: Applied Microservices for ShiftOps

Diagram


Client tier
HR admin
Dept manager
Staff member
Auditor
API gateway
Tenant resolution · JWT validation · Rate limiting · Routing · Logging
Auth service
JWT · Roles
Tenant · Sessions
Scheduling service
OR-Tools · Shift gen.
Conflict det.
Compliance service
Labour Act 2006
Hard/soft · Override
Payroll summary
Hour aggregation
CSV / JSON export
publishes events
Redis event bus
SchedulePublished · OverrideApplied · ShiftChanged · ComplianceViolationDetected
Notification service
In-app · Email · Batch dispatch
Audit service
Immutable log · Override records · Audit trail
Data tier
PostgreSQL (schema-per-tenant)
hospital_dhaka_medical · hospital_square
hospital_chittagong_general
Redis
Event bus (pub/sub)
Result cache · Session tokens

6.1. System Architecture Diagram Description

The ShiftOps architecture is organized into three tiers — Client, Service, and Data — connected through well-defined interfaces.

Client Tier: Hospital staff access ShiftOps via a React + Vite web application. All four user roles — HR Administrator, Department Manager, Staff Member, and Compliance Auditor — interact through the same web client, with role-scoped views and permissions enforced server-side. All client requests are HTTPS and target the API Gateway exclusively. No client has direct access to any microservice.

Service Tier: The API Gateway is the single entry point for all traffic. It performs three functions on every incoming request:

- Extracts and validates the JWT from the Authorization header by calling the Auth Service
- Parses the tenant_id and role claims from the validated token and injects them as request headers for downstream services
- Routes the request to the appropriate service based on the URL path prefix

Downstream services operate in two communication modes:

- **Synchronous REST** — API Gateway to Auth Service, Scheduling Service, Compliance Service, and Payroll Summary Service. These operations require an immediate response within the same HTTP request cycle.
- **Asynchronous Redis Pub/Sub** — Scheduling Service publishes events (SchedulePublished, OverrideApplied) to the Redis event bus. The Notification Service and Audit Service subscribe to these events and process them independently, without blocking the Scheduling Service's response to the manager.

Data Tier: All persistent data is stored in PostgreSQL using schema-per-tenant isolation. Each hospital's data resides in its own named schema. Redis serves two roles: the message broker for async inter-service events, and the caching layer for compliance validation results and session tokens.

6.2. Component Breakdown (Applied to ShiftOps)

Service	Function in Our Software

API Gateway	Intercepts all incoming HTTPS requests. Validates JWTs via the Auth Service. Extracts tenant_id and role from the token and injects them as headers into all downstream requests. Applies per-tenant rate limiting based on subscription plan. Routes requests to the correct service based on URL path prefix. Rejects requests from tenants with inactive subscriptions.
Auth Service	Issues signed JWTs on successful login, containing tenant_id, user_id, role, and expiry claims. Validates token signatures on request from the API Gateway. Manages user accounts and role assignments within each tenant. Enforces role hierarchy: HR Admin can authorize overrides; Department Manager can request them; Staff Members and Compliance Auditors have read-only access to schedules.
Scheduling Service	Accepts shift generation requests containing department, date range, required headcount per shift type, and staff availability. Submits the constraint model to the OR-Tools solver and receives an optimized schedule. Calls the Compliance Service synchronously for pre-publication validation. On receiving PASS, writes the schedule to the database and emits a SchedulePublished event. On SOFT_VIOLATION, returns the violation detail to the client for override authorization. On HARD_VIOLATION, rejects the operation.
Compliance Service	Validates proposed schedules against the constraint set. Hard constraints (no double-booking, no scheduling during approved leave, minimum headcount) always return HARD_VIOLATION with no override path. Soft constraints (48-hour weekly maximum, 11-hour rest minimum, 6-day consecutive maximum) return SOFT_VIOLATION with the violated rule detail and the role required to authorize an override. Returns PASS when all constraints are satisfied.
Notification Service	Subscribes to SchedulePublished and OverrideApplied events on the Redis bus. On SchedulePublished: groups affected staff by department and sends batched in-app and email notifications with the schedule link. On OverrideApplied: sends a targeted notification to the affected staff member with the override reason. Processes all notifications asynchronously — the publishing event does not wait for delivery confirmation.

Payroll Summary Service	Aggregates completed shift records for a specified pay period per employee. Calculates regular hours, overtime hours (above 48 hours/week as per Bangladesh Labour Act 2006 Section 108), and night shift allowances based on configured rates. Exports a structured JSON or CSV file for integration with the hospital's existing payroll system. Does not process payroll — acts as the data preparation and integration boundary only.
Audit Service	Subscribes to all system events from the Redis bus: schedule creation, publication, compliance violations, override applications, login events, and configuration changes. Writes immutable audit records to the audit_log table in the tenant's schema. Records cannot be edited or deleted. Exposes a read API for Compliance Auditors to query the audit trail by date range, event type, department, or user. Supports export as PDF or CSV for regulatory submission.

---

7. How the Pattern Works in Our Case

The Microservices pattern in ShiftOps produces three distinct interaction models, each reflecting a different aspect of how the architecture serves the hospital domain.

Synchronous request-response is used for all operations where a user is waiting for an immediate answer: logging in, viewing a schedule, requesting schedule generation, validating compliance before publication, and exporting a payroll summary. These flows traverse the API Gateway and one or two downstream services, completing within a single HTTP request cycle.

Asynchronous event-driven processing is used for all operations where the outcome does not need to be delivered in the same request cycle: notifying staff of a published schedule, logging audit records, and updating compliance dashboards. These flows are initiated by an event published to Redis and consumed independently by the Notification Service and Audit Service. The initiating service (Scheduling Service) does not wait for downstream processing to complete.

Override flow is a specialized hybrid: the authorization check is synchronous (the manager must wait for confirmation that the override is permitted), but the audit logging of the override is asynchronous (published as an OverrideApplied event). This design keeps the override confirmation fast while guaranteeing the audit record is always written.

Three key operational flows illustrate how the pattern works end-to-end:

Flow 1 — Schedule Generation and Publication:

- Department Manager submits a shift generation request for the ICU ward, two-week period

- API Gateway validates JWT, confirms Department Manager role, injects tenant_id, routes to Scheduling Service
- Scheduling Service submits constraint model to OR-Tools → receives optimized schedule draft
- Scheduling Service calls Compliance Service synchronously with the draft schedule
- Compliance Service validates all hard and soft constraints → returns PASS
- Scheduling Service writes schedule to the tenant's PostgreSQL schema
- Scheduling Service publishes SchedulePublished event to Redis
- Manager receives HTTP 200 success response — the request cycle ends here
- Notification Service consumes the event → batches and sends staff notifications
- Audit Service consumes the event → writes immutable publication record
- Both downstream operations complete asynchronously with no impact on the manager's response time

Flow 2 — Emergency Override:

- Manager attempts to assign an extra shift to a nurse already at 46 hours this week (proposed: 54 hours)
- Compliance Service detects soft constraint violation (48-hour maximum, Bangladesh Labour Act 2006 Section 100)
- Compliance Service returns SOFT_VIOLATION with rule detail and required authorization role: HR Administrator
- System presents the violation to the HR Administrator with a mandatory reason field and incident reference field
- HR Administrator enters reason ("Mass casualty event, all available staff required") and incident reference
- Auth Service confirms the user has HR Administrator role for the tenant
- Scheduling Service applies the assignment with the override payload
- Audit Service receives OverrideApplied event → logs: authorizing user, timestamp, violated rule, reason, incident reference
- Affected nurse receives notification that their schedule has been updated, with the override reason visible
- Compliance Auditor can query this override in the audit trail by incident reference or date

Flow 3 — New Hospital Onboarding:

- A new hospital completes the ShiftOps registration form
- Tenant Management Service creates a new tenant record and generates a unique tenant_id
- Service automatically creates a new PostgreSQL schema (hospital_[name]) and applies the full migration set
- Default Bangladesh Labour Act 2006 compliance rules are seeded into the tenant's configuration
- Initial HR Administrator account is created with a temporary password
- Welcome notification is dispatched via the Notification Service
- Entire process completes within 60 seconds — no manual intervention, no redeployment, no impact on existing tenants

---

7.1. Pattern Analysis

Overall Agility

Rating: High

Analysis: ShiftOps achieves high agility through the combination of independent service deployment and configuration-driven compliance rules. Each service can be updated, rolled back, or replaced without coordinating with other services. New product features are deployed behind feature flags, enabling gradual rollout without full redeployment. The most business-critical agility scenario — updating labor law compliance rules — requires only a configuration change with no code redeployment at all. The seven-service decomposition means that changes to any one domain (e.g., the payroll export format, the notification template engine, the OR-Tools solver version) are fully isolated from all other domains. Teams can work on different services in parallel without merge conflicts or deployment coordination.

Ease of Deployment

Rating: High

Analysis: Each of the seven services has its own CI/CD pipeline, container image, and deployment unit. A compliance rule update, a scheduling algorithm improvement, and a notification template change can all be deployed simultaneously by different team members without any coordination. Blue-green deployment per service ensures zero-downtime releases. Rollback is a one-command operation that reverts the container image and restores the load balancer to the previous version within 60 seconds. The expand-contract migration pattern ensures database changes are backward-compatible across service versions, allowing old and new versions of a service to run simultaneously during the transition window. Runtime tenant provisioning means new hospital onboarding requires no deployment activity at all.

Scalability

Rating: High

Analysis: ShiftOps achieves both horizontal and elastic scaling per service. The Scheduling Engine — the most computationally expensive service — scales to zero during idle periods and scales up on-demand for batch optimization jobs, independent of all other services. The Notification Service scales horizontally after a mass schedule publication event where thousands of staff notifications must be dispatched simultaneously. The Audit Service runs on minimal fixed resources because audit logging is low-latency write-only with no complex computation. Read-heavy schedule view queries scale independently via the PostgreSQL read replica, preventing read load from competing

with write operations on the primary database. The schema-per-tenant model means adding new hospital tenants does not add contention to existing tenants' data access paths.

Performance (Latency & Throughput)

Rating: Medium-High

Analysis: Synchronous operations (schedule viewing, compliance validation, authentication) complete within 500ms under normal load, meeting the performance scenario defined in the Utility Tree. The hybrid communication model — synchronous REST for request-response paths, asynchronous Redis Pub/Sub for notifications and audit logging — prevents expensive background operations from adding latency to the main request cycle. Compliance result caching eliminates redundant revalidation for unchanged schedules. The read replica routes schedule view queries away from the primary database. The one performance trade-off is the OR-Tools optimization: complex schedule generation for large departments can take 30–60 seconds. This is mitigated by running generation as an async batch job with a notification callback, so managers are not blocked waiting for the result. Cold start latency when the Scheduling Engine scales from zero (5–15 seconds) is a known trade-off accepted in exchange for energy efficiency during idle periods.

Maintainability

Rating: High

Analysis: Each service has a single, well-defined responsibility: the Compliance Service handles only compliance logic, the Audit Service handles only audit logging, the Scheduling Service handles only schedule management. This Single Responsibility principle at the service level means that when a compliance-related bug is reported, the debugging scope is limited to one service with one data model and one team owner. Inter-service contracts are versioned, so breaking changes in one service's API do not silently break dependent services. The configuration-driven compliance rules reduce the frequency of code changes required for regulatory updates. The seven services can be owned by as few as two developers in a course context, but the architecture scales to dedicated service teams without restructuring.

Energy Efficiency

Rating: High

Analysis: ShiftOps applies three complementary energy efficiency strategies. First, elastic scaling: the most expensive component (OR-Tools solver) runs only when needed and scales to zero during idle periods. Second, batch scheduling: full department schedule generation runs at off-peak hours

(2 AM), concentrating expensive computation during minimum-traffic periods when shared infrastructure is underutilized. Third, caching: compliance validation results are cached in Redis with schedule version keys, eliminating redundant recomputation for unchanged schedules. Asynchronous audit logging prevents CPU time from being consumed on write operations in the main request thread. Shared service instances across all tenants eliminate the idle resource waste of per-tenant deployments. Combined, these tactics reduce both cloud infrastructure cost and energy consumption relative to a monolithic always-on architecture.

Security & Confidentiality

Rating: High

Analysis: ShiftOps enforces security at multiple independent layers. Tenant isolation is structural: each hospital's data resides in a separate PostgreSQL schema, enforced at the database level independent of application logic. A bug in the application cannot accidentally expose cross-tenant data because the schema boundary exists at the infrastructure layer. JWT-based authentication with tenant-scoped role claims ensures that every request carries its authorization context, validated at the API Gateway before reaching any downstream service. The override-with-audit pattern ensures that every bypass of a compliance constraint is authorized, documented, and permanently recorded. Hard constraints cannot be overridden by any role, preventing the most serious violations regardless of user privilege level. The Audit Service's immutable log provides a non-repudiable record of all actions for regulatory review.

Availability

Rating: High

Analysis: ShiftOps meets the 99.93% uptime target for core scheduling functions through four complementary availability mechanisms. Fault isolation via the bulkhead pattern ensures that a failure in any single service (including non-critical services like Payroll Summary) cannot propagate to the Scheduling Service or Auth Service. Automatic container restart recovers transient failures within 30 seconds without manual intervention. Redundant active instances for the API Gateway and Scheduling Service mean that a single instance failure triggers immediate rerouting with no user-visible interruption. Graceful degradation ensures that even if the OR-Tools optimization engine is offline, staff and managers can continue to view published schedules in read-only mode, maintaining operational continuity during a partial outage. The Saga pattern with compensating transactions prevents partial failures from leaving the system in an inconsistent state.

Testability

Rating: High

Analysis: The microservices decomposition of ShiftOps enables multiple independent testing strategies. Each service can be unit-tested in complete isolation by injecting mock implementations of its dependencies — the Compliance Service can be tested against mock schedule data without a running Scheduling Service, database, or Redis instance. Inter-service contracts are defined as versioned REST APIs with explicit request/response schemas, enabling contract testing to catch breaking changes before deployment. The event-driven flows (notification dispatch, audit logging) can be tested by publishing mock events to a test Redis instance and verifying consumer behavior independently. The schema-per-tenant model allows integration tests to run against an isolated test schema without affecting production data. End-to-end tests can exercise the complete flow — from schedule generation through compliance validation to audit logging — using a dedicated test tenant with known seed data.

Conclusion

ShiftOps demonstrates that the Microservices architectural pattern is a direct, justified response to the specific operational demands of a hospital workforce scheduling platform — not a default architectural choice.

The availability analysis identified fault isolation, automatic recovery, and graceful degradation as critical requirements. These are satisfied structurally by the microservices deployment model, where each service is an independent failure domain. No amount of careful coding in a monolithic architecture can replicate structural fault isolation.

The deployability analysis identified independent compliance rule deployment and runtime tenant onboarding as critical requirements. Both are directly enabled by microservices: the Compliance Service deploys independently, and the Tenant Management Service provisions new schemas at runtime. These requirements are practically impossible to satisfy in a monolithic or layered architecture without accepting full system redeployment on every change.

The energy efficiency analysis identified elastic scaling of the OR-Tools solver, off-peak batch scheduling, and result caching as the primary efficiency levers. These require per-service resource allocation — a capability that only exists in a microservices deployment model where each service has independent scaling configuration.

The hard/soft constraint model and the override-with-audit pattern add architectural depth that elevates ShiftOps beyond a standard scheduling system. By acknowledging that hospitals face genuine emergencies that require compliance overrides, and by handling those overrides with mandatory authorization, mandatory justification, and immutable audit logging, the architecture reflects the real operational reality of the hospital domain rather than a simplified textbook abstraction.

ShiftOps as designed is a realistic, commercially viable SaaS platform whose every architectural decision traces directly to a business requirement, a quality attribute scenario, or a documented trade-off. This is the fundamental standard of good software architecture.